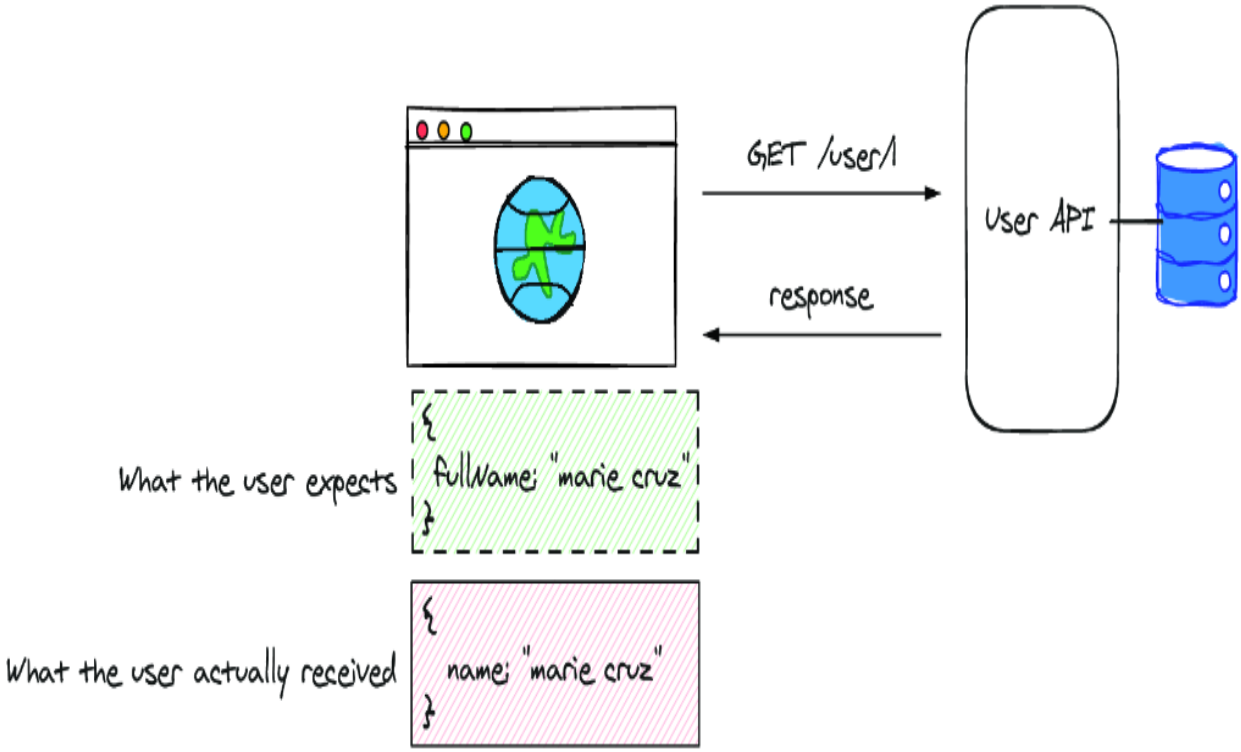


Figure 1.3 A web application communicating with the User API but with different expectations when it comes to the name key

At this point, the user accessing the web application won't be able to see their name, and there have been several complaints already. The worst-case scenario that could happen is that the production environment is broken until the team responsible for the User API either reverts the change or the team responsible for the web application updates their code, so they use the `name` key instead.

If both teams had contract testing, this breaking change would have been caught earlier, allowing the User API team to ask the web application team if the contract could be modified. If the web application team doesn't agree, then the breaking change needs to be reverted.

Another great reason why contract testing matters is that it gives data providers the confidence to introduce new requirements independently and safely. For example, let's imagine that apart from the web application team, the User API team also has another consumer. This other consumer, a mobile application, wants to display additional information about the user, such as the user's favorite items. The mobile application consumer creates a different contract detailing their expectations from the User API. The